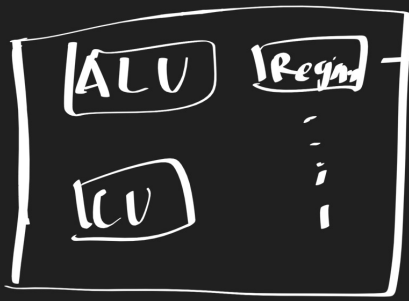
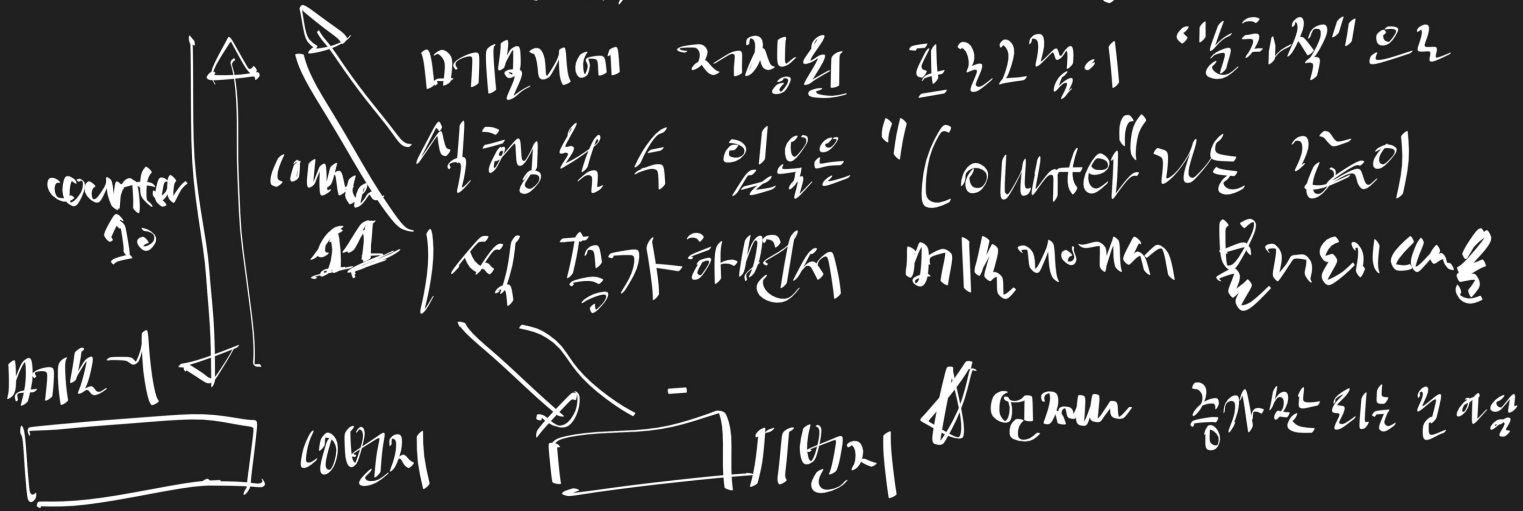


CPU



Register: 각기 다른 이름과 역할

① Program Counter (Instruction Counter)



② Instruction Register

: 해석할 명령어, 즉 메모리의 명령 가져온 명령어는 저장

메모리 → Program Counter → Instruction Register → ALU
↓ 지시어 선택

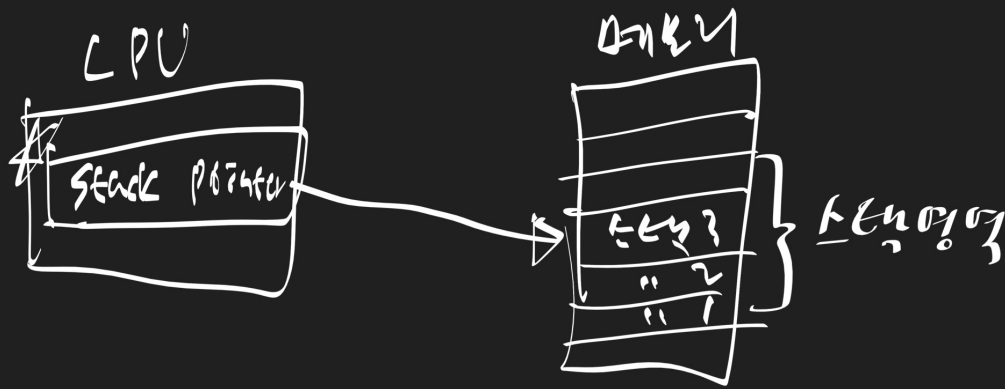
③ General Purpose Register

여러 역할 수행, 여러 프로그램 ~

④ Flag Register

연산의 결과, CPU 상태에 대한 복가 정보 Flag 저장

5) Stack Pointer



인터럽트 (interrupt)

: CPU의 작업을 방해하는 신호

① 전기 인터럽트: CPU가 예상치 못한 에너지를 수신함
aka "여외"

② 타이밍 인터럽트: 주기 입력장치에 의해 발생.
aka "와킹" & "하드웨어 인터럽트"

하드웨어 인터럽트

= CPU는 일을 하므로 병행처리를 수행하기 위하여

① 입력장치 장치는 CPU에게 "인터럽트 요청 신호"를 보냄 $\longleftrightarrow$ 폴링기법
: 리스틱 계속인

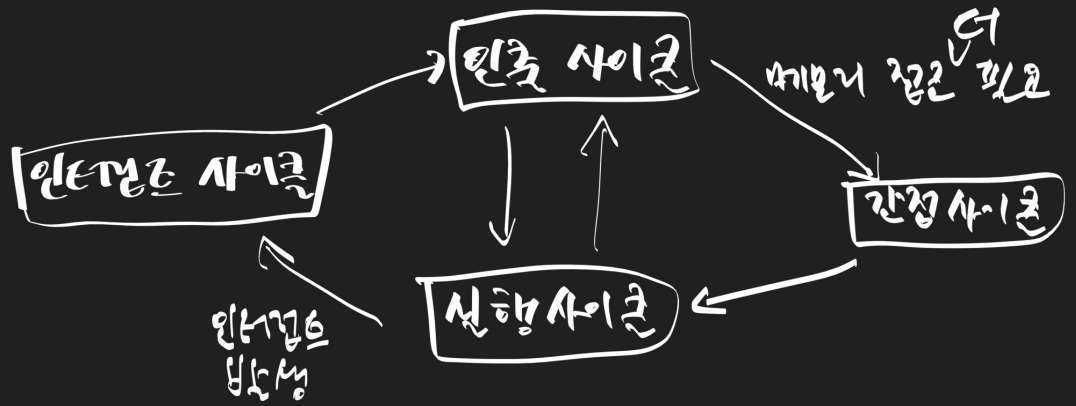
② CPU는 다른 실행이 끝나고 항상 인터럽트 여부 확인

③ 인터럽트 확인, "인터럽트 폴링"을 통해 바이트까지 여부 확인

④ If OK, 지금까지의 CPU 작업 백업

⑤ "인터럽트 벡터"를 사용하여 인터럽트 서비스 routine

⑥ if it is done, go back ④에서 백업한 작업에서 77



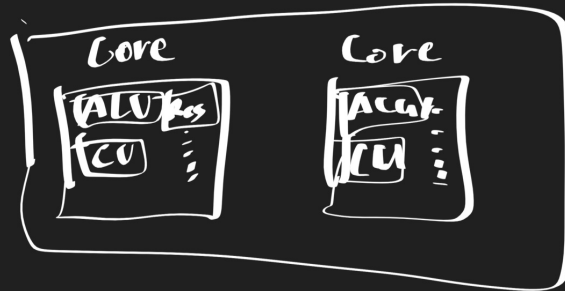
예외 (종기 인코딩)

- 출력 : 데이터가 발생하면 명령어부터 실행을 시작
- 입력 : " " 다음 명령어부터 " "
- 중단 : 프로그램 강제 종산 (abort)

CPU

클럭속도 : Hz 표기, CPU의 속도! 중요!

코어 : 인코딩 CPU



⇒ 각각 코어
인코딩 프로그램

실행속도 : 인코딩 프로그램 속도 : 코어의 코어가 동시에 처리하는
실행속도, 속도

병렬성 (Parallelism) : 작업을 여러개로 동시에 처리

동시성 (Concurrency) : 동시에 작업

병렬성 (Parallelism) : 작업을 물리적으로 동시에 처리

동시성 (Concurrency) : 동시에 작업을 처리하는 것 처럼 보임
CPU가 하나씩 번갈아가면서 작업.

Task 1
Task 2
Task 3

동시성
task 1 - - -
task 2 - - -
task 3 - - -

★ 파이프라이닝을 통한 병렬성 향상시키기

★ Instruction-level Parallelism (병렬어 명령 처리 방법)

: 여러 명령어를 동시에 처리하여
CPU를 한시로 수직적으로 활용하는 방법!

- ① Instruction Fetch CPU가 각각의 단계를 동시에 실행할 수 있음
- ② Instruction Decode
- ③ Instruction Execute
- ④ Write back

★ Superscalar : CPU 내부에 여러 명령어 파이프라인 포함하는 시스템

★ Pipeline hazard :

- 데이터 위험 - 데이터 의존성에 의해.
- 제어 위험 - 프로그램 카운터의 갑작스런 변위로
- 구조적 위험 (자원 위험) - 다른 명령어가 같은 리소스 사용